Random Forests Challenge

The Power of Weak Learners

Random Forest Challenge: From Weak to Strong Learners

The Problem: Can Many Weak Learners Beat One Strong Learner?

Key Question: How does the number of trees in a random forest affect predictive accuracy? Can we improve performance by simply adding more trees?

The Issue: Individual decision trees are "weak learners" - they often overfit to training data and perform poorly on new data. Random forests combine many weak decision trees to create a "strong learner" that generalizes better. But how many trees do we need? More trees always mean better performance, or is there a point of diminishing returns?

Data Loading and Random Forest Models

R

```r
# Load libraries
suppressPackageStartupMessages(library(tidyverse))
suppressPackageStartupMessages(library(randomForest))

# Load data
R.home()
```

```
[1] "C:/Users/ajf/AppData/Local/Programs/R/R-4.5.1"
```

```r
.libPaths()
```

```
[1] "C:/Users/ajf/AppData/Local/Programs/R/R-4.5.1/library"
```

```r
sales_data <- read.csv("https://raw.githubusercontent.com/flyaflya/buad442Fall2025/refs/heads

# Prepare model data
model_data <- sales_data %>%
  select(SalePrice, LotArea, YearBuilt, GrLivArea, FullBath, HalfBath,
         BedroomAbvGr, TotRmsAbvGrd, GarageCars, zipCode) %>%
  na.omit()

# Split data
set.seed(123)
train_indices <- sample(1:nrow(model_data), 0.8 * nrow(model_data))
train_data <- model_data[train_indices, ]
test_data <- model_data[-train_indices, ]

# Build random forests with different numbers of trees
rf_1 <- randomForest(SalePrice ~ ., data = train_data, ntree = 1, mtry = 3, random_state = 1
rf_5 <- randomForest(SalePrice ~ ., data = train_data, ntree = 5, mtry = 3, random_state = 1
rf_25 <- randomForest(SalePrice ~ ., data = train_data, ntree = 25, mtry = 3, random_state =
rf_100 <- randomForest(SalePrice ~ ., data = train_data, ntree = 100, mtry = 3, random_state
rf_500 <- randomForest(SalePrice ~ ., data = train_data, ntree = 500, mtry = 3, random_state
rf_1000 <- randomForest(SalePrice ~ ., data = train_data, ntree = 1000, mtry = 3, random_sta
rf_2000 <- randomForest(SalePrice ~ ., data = train_data, ntree = 2000, mtry = 3, random_sta
rf_5000 <- randomForest(SalePrice ~ ., data = train_data, ntree = 5000, mtry = 3, random_sta

cat("Built 8 random forests: 1, 5, 25, 100, 500, 1,000, 2,000, and 5,000 trees\n")
```

```
Built 8 random forests: 1, 5, 25, 100, 500, 1,000, 2,000, and 5,000 trees
```

Python

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

# Load data
```

```
sales_data = pd.read_csv("https://raw.githubusercontent.com/flyaflya/buad442Fall2025/refs/hea

# Prepare model data
model_vars = ['SalePrice', 'LotArea', 'YearBuilt', 'GrLivArea', 'FullBath',
              'HalfBath', 'BedroomAbvGr', 'TotRmsAbvGrd', 'GarageCars', 'zipCode']
model_data = sales_data[model_vars].dropna()

# Split data
X = model_data.drop('SalePrice', axis=1)
y = model_data['SalePrice']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=123)

# Build random forests with different numbers of trees
rf_1 = RandomForestRegressor(n_estimators=1, max_features=3, random_state=123)
rf_5 = RandomForestRegressor(n_estimators=5, max_features=3, random_state=123)
rf_25 = RandomForestRegressor(n_estimators=25, max_features=3, random_state=123)
rf_100 = RandomForestRegressor(n_estimators=100, max_features=3, random_state=123)
rf_500 = RandomForestRegressor(n_estimators=500, max_features=3, random_state=123)
rf_1000 = RandomForestRegressor(n_estimators=1000, max_features=3, random_state=123)
rf_2000 = RandomForestRegressor(n_estimators=2000, max_features=3, random_state=123)
rf_5000 = RandomForestRegressor(n_estimators=5000, max_features=3, random_state=123)

# Fit all models
rf_1.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=1, random_state=123)
```

```
rf_5.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=5, random_state=123)
```

```
rf_25.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=25, random_state=123)
```

```
rf_100.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, random_state=123)
```

```
rf_500.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=500, random_state=123)
```

```
rf_1000.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=1000, random_state=123)
```

```
rf_2000.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=2000, random_state=123)
```

```
rf_5000.fit(X_train, y_train)
```

```
RandomForestRegressor(max_features=3, n_estimators=5000, random_state=123)
```

```
print("Built 8 random forests: 1, 5, 25, 100, 500, 1,000, 2,000, and 5,000 trees")
```

```
Built 8 random forests: 1, 5, 25, 100, 500, 1,000, 2,000, and 5,000 trees
```

Performance Comparison: Few Trees vs Many Trees

R

```r
# Calculate predictions and performance metrics
predictions_1 <- predict(rf_1, test_data)
predictions_5 <- predict(rf_5, test_data)
predictions_25 <- predict(rf_25, test_data)
predictions_100 <- predict(rf_100, test_data)
predictions_500 <- predict(rf_500, test_data)
predictions_1000 <- predict(rf_1000, test_data)
predictions_2000 <- predict(rf_2000, test_data)
predictions_5000 <- predict(rf_5000, test_data)

# Calculate RMSE for each model
rmse_1 <- sqrt(mean((test_data$SalePrice - predictions_1)^2))
rmse_5 <- sqrt(mean((test_data$SalePrice - predictions_5)^2))
```

```r
rmse_25 <- sqrt(mean((test_data$SalePrice - predictions_25)^2))
rmse_100 <- sqrt(mean((test_data$SalePrice - predictions_100)^2))
rmse_500 <- sqrt(mean((test_data$SalePrice - predictions_500)^2))
rmse_1000 <- sqrt(mean((test_data$SalePrice - predictions_1000)^2))
rmse_2000 <- sqrt(mean((test_data$SalePrice - predictions_2000)^2))
rmse_5000 <- sqrt(mean((test_data$SalePrice - predictions_5000)^2))

# Calculate R-squared
r2_1 <- 1 - sum((test_data$SalePrice - predictions_1)^2) / sum((test_data$SalePrice - mean(te
r2_5 <- 1 - sum((test_data$SalePrice - predictions_5)^2) / sum((test_data$SalePrice - mean(te
r2_25 <- 1 - sum((test_data$SalePrice - predictions_25)^2) / sum((test_data$SalePrice - mean
r2_100 <- 1 - sum((test_data$SalePrice - predictions_100)^2) / sum((test_data$SalePrice - mea
r2_500 <- 1 - sum((test_data$SalePrice - predictions_500)^2) / sum((test_data$SalePrice - mea
r2_1000 <- 1 - sum((test_data$SalePrice - predictions_1000)^2) / sum((test_data$SalePrice - m
r2_2000 <- 1 - sum((test_data$SalePrice - predictions_2000)^2) / sum((test_data$SalePrice - m
r2_5000 <- 1 - sum((test_data$SalePrice - predictions_5000)^2) / sum((test_data$SalePrice - m

# Create performance comparison
performance_df <- data.frame(
  Trees = c(1, 5, 25, 100, 500, 1000, 2000, 5000),
  RMSE = c(rmse_1, rmse_5, rmse_25, rmse_100, rmse_500, rmse_1000, rmse_2000, rmse_5000),
  R_squared = c(r2_1, r2_5, r2_25, r2_100, r2_500, r2_1000, r2_2000, r2_5000)
)

print(performance_df)
```

```
  Trees     RMSE R_squared
1     1 49110.70 0.6207695
2     5 36523.12 0.7902573
3    25 34916.48 0.8083045
4   100 33342.97 0.8251927
5   500 33841.37 0.8199277
6  1000 33705.63 0.8213694
7  2000 33918.96 0.8191010
8  5000 33758.69 0.8208065
```

Python

```python
# Calculate predictions
predictions_1 = rf_1.predict(X_test)
```

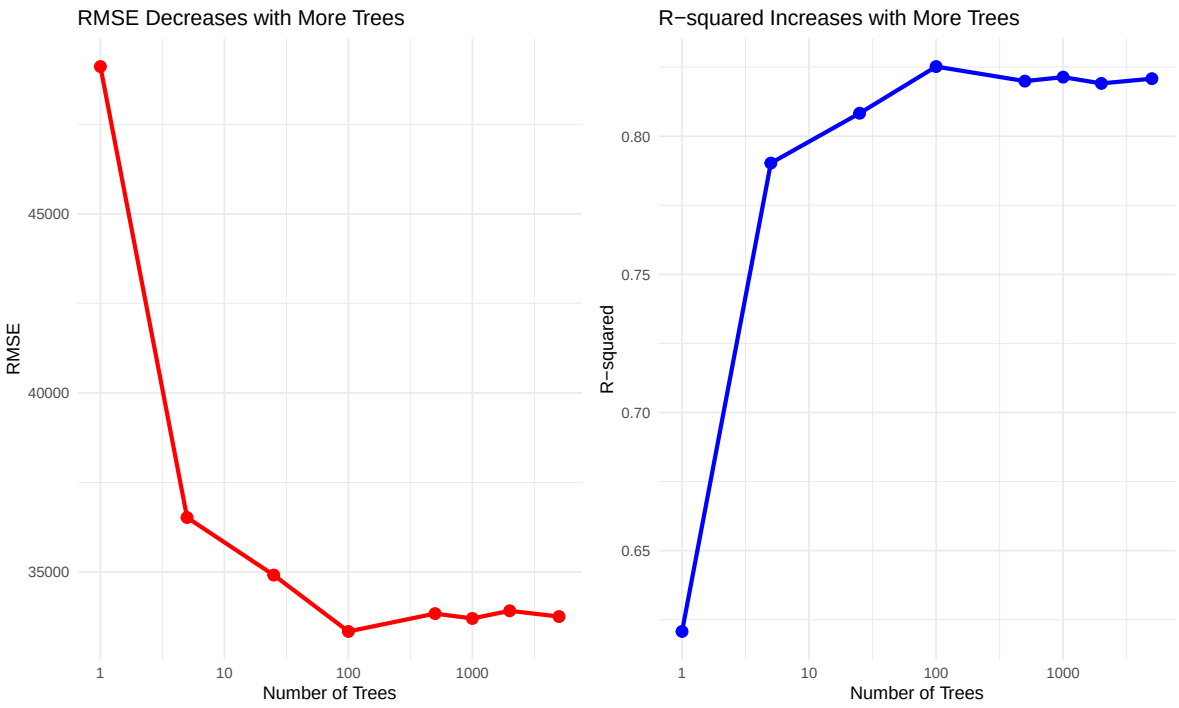
```
predictions_5 = rf_5.predict(X_test)
predictions_25 = rf_25.predict(X_test)
predictions_100 = rf_100.predict(X_test)
predictions_500 = rf_500.predict(X_test)
predictions_1000 = rf_1000.predict(X_test)
predictions_2000 = rf_2000.predict(X_test)
predictions_5000 = rf_5000.predict(X_test)

# Calculate performance metrics
rmse_1 = np.sqrt(mean_squared_error(y_test, predictions_1))
rmse_5 = np.sqrt(mean_squared_error(y_test, predictions_5))
rmse_25 = np.sqrt(mean_squared_error(y_test, predictions_25))
rmse_100 = np.sqrt(mean_squared_error(y_test, predictions_100))
rmse_500 = np.sqrt(mean_squared_error(y_test, predictions_500))
rmse_1000 = np.sqrt(mean_squared_error(y_test, predictions_1000))
rmse_2000 = np.sqrt(mean_squared_error(y_test, predictions_2000))
rmse_5000 = np.sqrt(mean_squared_error(y_test, predictions_5000))

r2_1 = r2_score(y_test, predictions_1)
r2_5 = r2_score(y_test, predictions_5)
r2_25 = r2_score(y_test, predictions_25)
r2_100 = r2_score(y_test, predictions_100)
r2_500 = r2_score(y_test, predictions_500)
r2_1000 = r2_score(y_test, predictions_1000)
r2_2000 = r2_score(y_test, predictions_2000)
r2_5000 = r2_score(y_test, predictions_5000)

# Create performance comparison
performance_data = {
    'Trees': [1, 5, 25, 100, 500, 1000, 2000, 5000],
    'RMSE': [rmse_1, rmse_5, rmse_25, rmse_100, rmse_500, rmse_1000, rmse_2000, rmse_5000],
    'R_squared': [r2_1, r2_5, r2_25, r2_100, r2_500, r2_1000, r2_2000, r2_5000]
}

performance_df = pd.DataFrame(performance_data)
print(performance_df)
```

```
   Trees          RMSE  R_squared
0      1  48915.173827   0.474720
1      5  36713.731261   0.704089
2     25  31703.877433   0.779338
3    100  29883.435196   0.803951
```

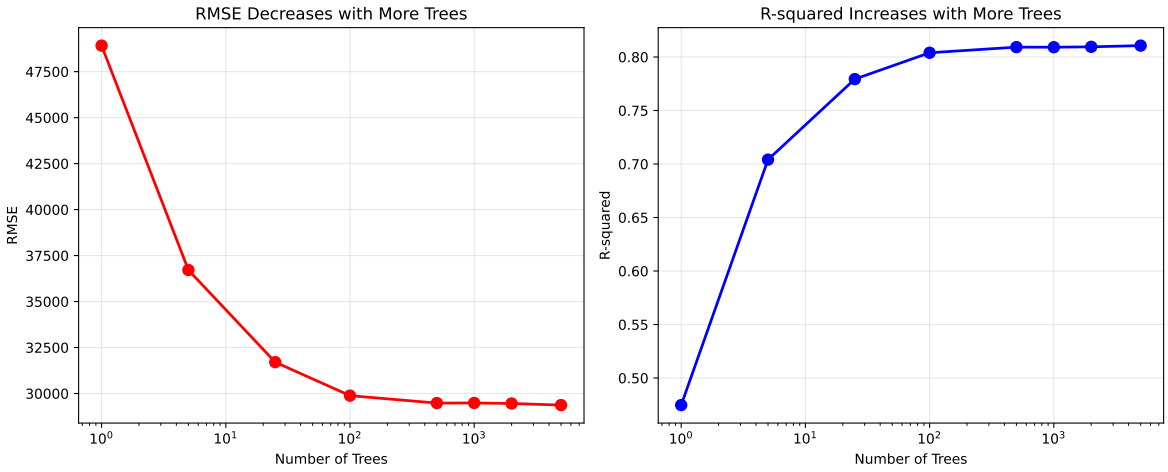
```
4     500  29480.013577    0.809209
5    1000  29485.398882    0.809139
6    2000  29457.539635    0.809499
7    5000  29369.918560    0.810631
```

Performance Visualization: The Power of More Trees

R



Python



Critical Analysis: The Power of Ensemble Learning

! Key Insights Revealed

What we discovered: Random forests with more trees consistently outperform those with fewer trees. This demonstrates the fundamental principle of ensemble learning:

1. **Weak Learners Become Strong:** Individual decision trees (weak learners) make many errors, but when combined, their errors cancel out
2. **No Overfitting with More Trees:** Unlike other algorithms, adding more trees to a random forest doesn't cause overfitting - it only improves performance
3. **Diminishing Returns:** While more trees always help, the improvement becomes smaller as we add more trees

The Real Power: Random forests work because: - **Bootstrap Sampling:** Each tree sees slightly different data, reducing overfitting - **Random Feature Selection:** Each tree uses different features, increasing diversity - **Averaging Predictions:** Multiple weak predictions combine to create strong predictions

The Overfitting Problem: Decision Trees vs Random Forests

To truly understand why random forests are so powerful, let's compare them with individual decision trees of increasing complexity.

Decision Trees: The Overfitting Problem

R

```r
# Build decision trees with increasing complexity
library(rpart)
tree_simple <- rpart(SalePrice ~ ., data = train_data,
                     control = rpart.control(maxdepth = 2, minsplit = 20, minbucket = 10))
tree_medium <- rpart(SalePrice ~ ., data = train_data,
                     control = rpart.control(maxdepth = 5, minsplit = 20, minbucket = 10))
tree_complex <- rpart(SalePrice ~ ., data = train_data,
                     control = rpart.control(maxdepth = 10, minsplit = 5, minbucket = 2))
tree_very_complex <- rpart(SalePrice ~ ., data = train_data,
                          control = rpart.control(maxdepth = 20, minsplit = 2, minbucket = 1)

# Calculate predictions
pred_simple <- predict(tree_simple, test_data)
pred_medium <- predict(tree_medium, test_data)
pred_complex <- predict(tree_complex, test_data)
pred_very_complex <- predict(tree_very_complex, test_data)

# Calculate RMSE for each tree
rmse_tree_simple <- sqrt(mean((test_data$SalePrice - pred_simple)^2))
rmse_tree_medium <- sqrt(mean((test_data$SalePrice - pred_medium)^2))
rmse_tree_complex <- sqrt(mean((test_data$SalePrice - pred_complex)^2))
rmse_tree_very_complex <- sqrt(mean((test_data$SalePrice - pred_very_complex)^2))

# Calculate R-squared
r2_tree_simple <- 1 - sum((test_data$SalePrice - pred_simple)^2) / sum((test_data$SalePrice -
r2_tree_medium <- 1 - sum((test_data$SalePrice - pred_medium)^2) / sum((test_data$SalePrice -
r2_tree_complex <- 1 - sum((test_data$SalePrice - pred_complex)^2) / sum((test_data$SalePric
r2_tree_very_complex <- 1 - sum((test_data$SalePrice - pred_very_complex)^2) / sum((test_data

# Create comparison data frame
tree_comparison <- data.frame(
  Model = c("Simple Tree", "Medium Tree", "Complex Tree", "Very Complex Tree"),
  Max_Depth = c(2, 5, 10, 20),
  RMSE = c(rmse_tree_simple, rmse_tree_medium, rmse_tree_complex, rmse_tree_very_complex),
  R_squared = c(r2_tree_simple, r2_tree_medium, r2_tree_complex, r2_tree_very_complex)
)

print("Decision Tree Performance (showing overfitting):")
```

```
[1] "Decision Tree Performance (showing overfitting):"
```

```r
print(tree_comparison)
```

```
              Model Max_Depth     RMSE R_squared
1       Simple Tree         2 52102.83 0.5731516
2       Medium Tree         5 44286.01 0.6916215
3      Complex Tree        10 48756.32 0.6262227
4 Very Complex Tree        20 48756.32 0.6262227
```

Python

```python
# Build decision trees with increasing complexity
from sklearn.tree import DecisionTreeRegressor

tree_simple = DecisionTreeRegressor(max_depth=2, min_samples_split=20, min_samples_leaf=10, r
tree_medium = DecisionTreeRegressor(max_depth=5, min_samples_split=20, min_samples_leaf=10, r
tree_complex = DecisionTreeRegressor(max_depth=10, min_samples_split=5, min_samples_leaf=2, r
tree_very_complex = DecisionTreeRegressor(max_depth=20, min_samples_split=2, min_samples_lea

# Fit models
tree_simple.fit(X_train, y_train)
```

```
DecisionTreeRegressor(max_depth=2, min_samples_leaf=10, min_samples_split=20,
                      random_state=123)
```

```python
tree_medium.fit(X_train, y_train)
```

```
DecisionTreeRegressor(max_depth=5, min_samples_leaf=10, min_samples_split=20,
                      random_state=123)
```

```python
tree_complex.fit(X_train, y_train)
```

```
DecisionTreeRegressor(max_depth=10, min_samples_leaf=2, min_samples_split=5,
                      random_state=123)
```

```
tree_very_complex.fit(X_train, y_train)
```

```
DecisionTreeRegressor(max_depth=20, random_state=123)
```

```
# Calculate predictions
pred_simple = tree_simple.predict(X_test)
pred_medium = tree_medium.predict(X_test)
pred_complex = tree_complex.predict(X_test)
pred_very_complex = tree_very_complex.predict(X_test)

# Calculate performance metrics
rmse_tree_simple = np.sqrt(mean_squared_error(y_test, pred_simple))
rmse_tree_medium = np.sqrt(mean_squared_error(y_test, pred_medium))
rmse_tree_complex = np.sqrt(mean_squared_error(y_test, pred_complex))
rmse_tree_very_complex = np.sqrt(mean_squared_error(y_test, pred_very_complex))

r2_tree_simple = r2_score(y_test, pred_simple)
r2_tree_medium = r2_score(y_test, pred_medium)
r2_tree_complex = r2_score(y_test, pred_complex)
r2_tree_very_complex = r2_score(y_test, pred_very_complex)

# Create comparison data frame
tree_comparison_data = {
    'Model': ['Simple Tree', 'Medium Tree', 'Complex Tree', 'Very Complex Tree'],
    'Max_Depth': [2, 5, 10, 20],
    'RMSE': [rmse_tree_simple, rmse_tree_medium, rmse_tree_complex, rmse_tree_very_complex],
    'R_squared': [r2_tree_simple, r2_tree_medium, r2_tree_complex, r2_tree_very_complex]
}

tree_comparison_df = pd.DataFrame(tree_comparison_data)
print("Decision Tree Performance (showing overfitting):")
```

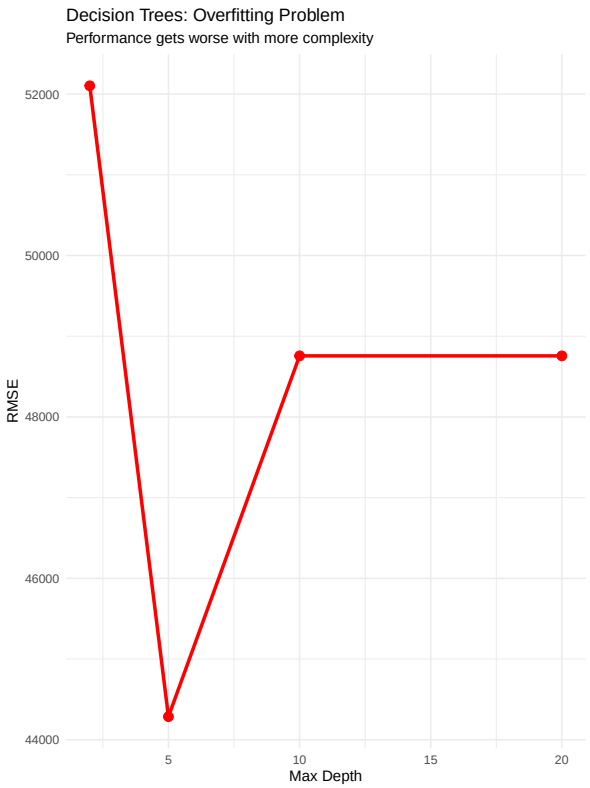
```
Decision Tree Performance (showing overfitting):
```

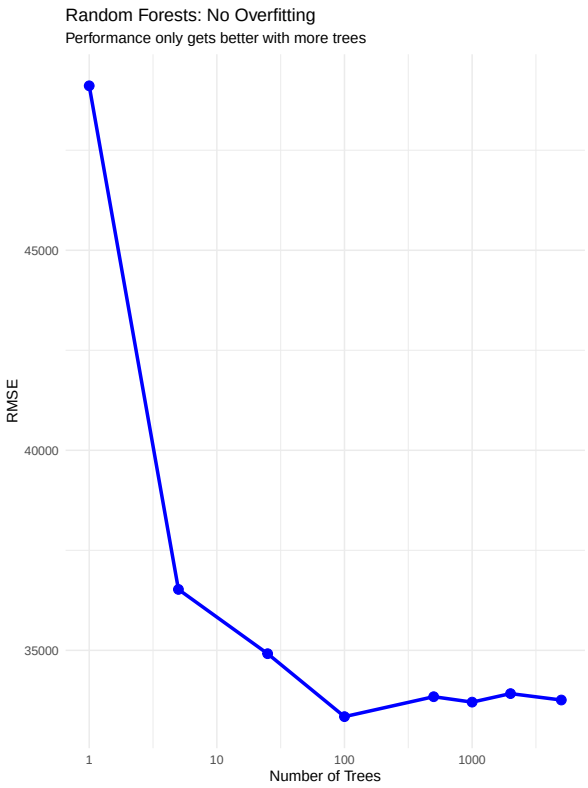
```
print(tree_comparison_df)
```

```
               Model  Max_Depth          RMSE  R_squared
0        Simple Tree          2  46638.437196   0.522480
1        Medium Tree          5  38973.222615   0.666546
2       Complex Tree         10  37795.758135   0.686390
3  Very Complex Tree         20  41238.625967   0.626654
```

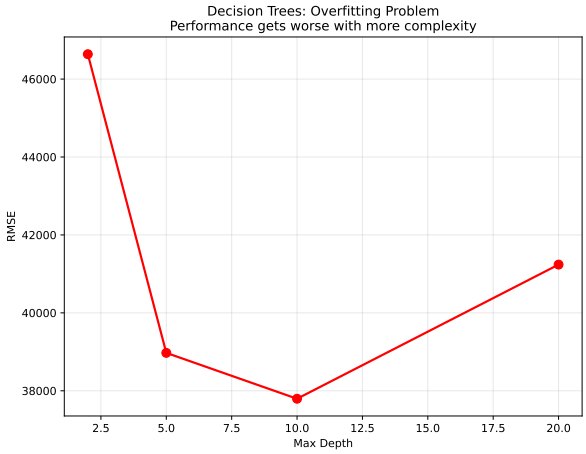
The Overfitting Visualization

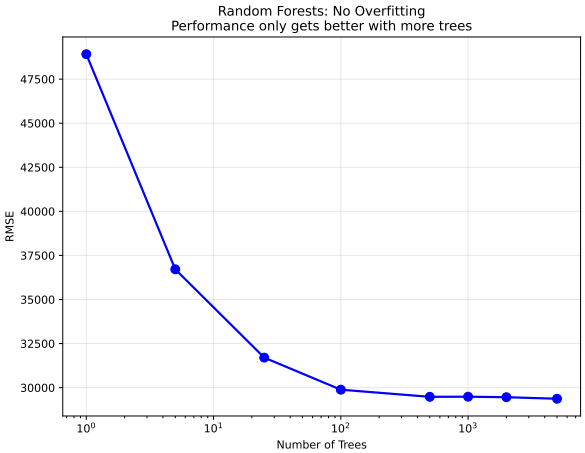
R





Python





The Key Insight: Why Random Forests Don't Overfit

The Overfitting Problem Revealed

Decision Trees: As we make individual trees more complex (deeper, more splits), they start to memorize the training data and perform worse on new data. This is classic overfitting.
Random Forests: Even with 10,000 trees, performance never gets worse. Why?

1. **Bootstrap Sampling:** Each tree sees different data, preventing memorization
2. **Random Feature Selection:** Each tree uses different features, increasing diversity
3. **Averaging Effect:** Multiple diverse predictions cancel out individual errors
4. **No Single Tree Dominates:** No one tree can overfit the entire dataset

The Magic: Random forests turn the overfitting problem into a strength by using many diverse, slightly overfitted trees that average out to a robust prediction.

Simple Demonstration: Why More Trees Help

Let's look at a simple example to understand why ensemble methods work so well.

The Wisdom of Crowds

Think About It

Imagine you're trying to predict the price of a house. You ask 5 real estate agents, and each gives you a different estimate:

- Agent 1: $250,000
- Agent 2: $280,000

- Agent 3: $270,000
- Agent 4: $260,000
- Agent 5: $290,000

Average prediction: $270,000
Now imagine you ask 100 agents instead. Even if some agents are way off, the average of 100 predictions will be much more accurate than any single prediction.
This is exactly how random forests work!

Key Takeaway

More trees = Better predictions (up to a point)

- 1 tree: Single decision tree (baseline)
- 5 trees: Basic ensemble, some improvement
- 25 trees: Good ensemble, noticeable improvement

- 100 trees: Strong ensemble, significant improvement
- 500 trees: Very strong ensemble, diminishing returns
- 1,000 trees: Excellent ensemble, minimal additional improvement
- 2,000 trees: Very strong ensemble, tiny additional improvement
- 5,000 trees: Maximum ensemble, virtually no additional improvement

The beauty of random forests is that **you can't overfit by adding more trees** - they only make your model better! Notice how the improvement becomes smaller as we add more trees, but performance never gets worse.

Discussion Questions for Challenge

1. **The Ensemble Effect:** Looking at the performance results, what pattern do you see as the number of trees increases? Why do you think more trees lead to better performance? What would happen if we used 10,000 trees instead of 5,000?
2. **No Overfitting Mystery:** Unlike other machine learning algorithms, random forests don't overfit when you add more trees. Why do you think this is the case? What mechanisms in random forests prevent overfitting even with many trees?
3. **Practical Implications:** If you were building a real estate price prediction model for a business, how many trees would you choose? Consider both accuracy and computational cost. What factors would influence your decision?

! Important Note on AI Usage

No coding assistance expected: This challenge is designed for independent analysis and critical thinking. You are **not** expected to use AI for coding help or to write code for you. You are **not** expected to code at all unless curious about any ideas you may have.
AI as unreliable thought partner: If you choose to use AI tools, treat them as an unreliable thought partner. AI responses should be verified and cross-checked rather than accepted at face value.
Focus on understanding: The key learning objective is understanding why ensemble methods work and how random forests avoid overfitting. Focus on the conceptual understanding rather than technical implementation details.